



OpenCode

AI coding agents in your terminal

An open-source, terminal-native coding agent. Connect it to e-INFRA models, add MCP tools, and build your own agents and subagents.

RSE

Roadmap

What we'll cover in this session

1

What is OpenCode

The tool, its architecture, and why it's different

2

Install & first run

Get the CLI, launch the TUI, basic navigation

3

Connect e-INFRA models

Point OpenCode at `llm.ai.e-infra.cz`

4

Add MCP servers

Plug in external tools, local and remote

5

Agents & subagents

Build focused assistants and delegate work

6

Sandboxing & permissions

Control what the agent is allowed to do

What Is OpenCode?

An open-source AI coding agent built for the terminal

Terminal-native

Runs as a fast TUI right in your shell. Also drives an IDE extension, a web UI, and a headless server / SDK.

Model-agnostic

Not tied to any one vendor. Use Anthropic, OpenAI, local models, or any OpenAI-compatible endpoint — like e-INFRA.

Open source

100% open, Apache-2.0. A client/server architecture means the same engine powers the TUI, IDE, and automation.

Extensible

Add MCP tools, custom agents and subagents, skills, commands, LSP servers, and plugins — all via simple config.

→ One agent, many surfaces — and you stay in control of the model, the tools, and the permissions.

Install & First Run

From zero to a running agent in your project

```
# Install (pick one)
curl -fsSL https://opencode.ai/install | bash
npm i -g opencode-ai
brew install sst/tap/opencode

# Launch in your project directory
cd my-project
opencode

# Inside the TUI
/connect      # add a model provider + API key
/models      # pick the active model
/agent       # switch agent (or press Tab)
@explore find the auth code # call a subagent
```

Handy keys & commands

Tab

Cycle primary agents

@name

Mention a subagent

/connect

Add a provider

/models

Choose a model

/share

Share a session

esc

Interrupt the agent

Connect to e-INFRA Models

Add the e-INFRA CZ provider to `~/.config/opencode/opencode.json`

```
// inside "provider": { ... }
"einfra": {
  "npm": "@ai-sdk/openai-compatible",
  "name": "e-INFRA CZ",
  "options": {
    "baseUrl": "https://llm.ai.e-infra.cz/v1",
    "apiKey": "{env:E_INFRA_API_TOKEN}"
  },
  "models": {
    "agentic": { "name": "agentic" },
    "coder": { "name": "coder" },
    "command-a": { "name": "command-a" },
    "deepseek-v4-pro": { "name": "deepseek-v4-pro" },
    "gemma4": { "name": "gemma4" },
    "glm-5.1": { "name": "glm-5.1" },
    "gpt-oss-120b": { "name": "gpt-oss-120b" },
    "kimi-k2.5": { "name": "kimi-k2.5" },
    "kimi-k2.6": { "name": "kimi-k2.6" }
  }
}
```

Then, inside opencode

- 1 Run `/connect` → pick e-INFRA CZ
- 2 Paste your API key from `chat.ai.e-infra.cz`
- 3 Run `/models` → pick a model (e.g. coder)

Aliases: `agentic · coder · mini · thinker` track current models — no need to edit config when they change.

Add MCP Servers

Give the agent external tools — local processes or remote services

Local (stdio)

A command OpenCode spawns and talks to over stdio.

```
"mcp": {
  "my-tools": {
    "type": "local",
    "command": ["npx", "-y", "my-mcp"],
    "environment": {
      "API_KEY": "${env:API_KEY}"
    },
    "enabled": true
  }
}
```

Remote (HTTP / SSE)

A hosted MCP endpoint, with headers or OAuth auth.

```
"mcp": {
  "context7": {
    "type": "remote",
    "url": "https://mcp.context7.com/mcp",
    "headers": {
      "API_KEY": "${env:CONTEXT7_KEY}"
    }
  }
}
```

! MCP tools are auto-discovered and added to the model's context — enable only what you need, big servers eat tokens fast.

Create a New Agent

Focused assistants with their own model, prompt, and permissions

Two flavors

Primary — main agents you drive directly (Build, Plan). Switch with Tab.

Subagent — specialists invoked by other agents or via @mention.

Scaffold one interactively:

```
opencode agent create
```

Options: description · mode · model · prompt · permission
· temperature

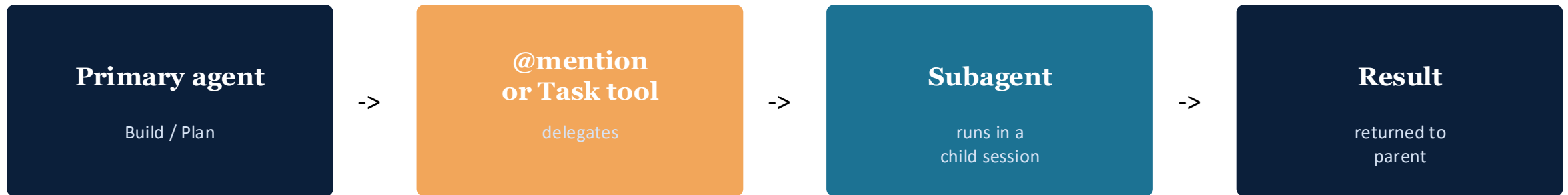
```
.opencode/agents/reviewer.md
```

```
---
description: Reviews code for bugs & security
mode: subagent
model: einfra/qwen3-coder
temperature: 0.1
permission:
  edit: deny
  bash: ask
---
```

You are a senior code reviewer. Focus on correctness, security, and clear feedback. Do not modify files – suggest changes only.

Subagents

Delegate specialized work into isolated child sessions



Built-in subagents

general	Multi-step research & tasks, full tools — run several in parallel
explore	Fast, read-only codebase search — never edits
scout	Read-only research of external docs & dependencies

Why use them

Parallelism — run many units of work at once.

Clean context — each child gets its own session, keeping the main thread focused.

Least privilege — scope tools/permissions per subagent; gate who can call whom with `permission.task`.

Sandboxing & Permissions

Decide what runs automatically, what asks first, and what's blocked

"allow"

Run without approval

Trusted, reversible actions

"ask"

Prompt before running

The safe default for risky tools

"deny"

Block entirely

Tools the agent must never use

```
"permission": {
  "*": "ask",
  "edit": "allow",
  "bash": {
    "*": "ask",
    "git *": "allow",
    "rm *": "deny"
  }
}
```

Built-in guardrails

- .env files are denied by default
- Leaving the project dir → external_directory asks
- Repeated identical calls trip the doom_loop guard
- Override any rule per agent (e.g. Plan = read-only)



Takeaways

One terminal, any model

- Open-source TUI coding agent
- e-INFRA via an OpenAI-compatible provider

Extend with MCP

- Local (stdio) and remote (HTTP) tools
- Auto-added to context — enable only what you need

Agents & subagents

- Primary agents you drive directly
- Subagents for parallel, scoped work
- Markdown or JSON, or agent create

Stay in control

- allow / ask / deny per tool & command
- .env and external dirs guarded by default
- Override permissions per agent

The pitch: *your shell, your models, your tools — with permissions you set.*